

Progetto B5: Storage chiave-valore distribuito con consenso, snapshot e deployment su Amazon EC2

Enrico Barbatano

Università degli Studi di Roma Tor Vergata
Corso di Sistemi Distribuiti e Cloud Computing
Email: enrico.barbatano@students.uniroma2.eu

Sommario—Questo elaborato presenta uno storage chiave-valore distribuito sviluppato in Go per il progetto B5. Il sistema elimina la dipendenza da un database centralizzato replicando lo stato tramite un protocollo basato su log replicato e ispirato a Raft, con elezione del leader, commit su quorum e persistenza locale tramite WAL e snapshot. L'architettura include Consensus Node stateful, un Client Proxy stateless per la discovery dinamica del leader e un Backup Service stateless rispetto al consenso e allo stato applicativo, usato come gestore esterno di snapshot persistenti. La comunicazione usa gRPC e Protocol Buffers, mentre Client-side Service Discovery e Circuit Breaker migliorano trasparenza e resilienza. Il sistema è stato containerizzato con Docker Compose, validato tramite CI e distribuito su Amazon EC2. I benchmark end-to-end mostrano success rate del 100% con 3, 5 e 7 nodi, throughput vicino a 9 ops/s, rielezione del leader in circa 1.27 s in media e backup parziale completato con 4 snapshot su 5 in presenza di un nodo offline.

Index Terms—Sistemi distribuiti, consenso, Raft, gRPC, Docker Compose, Amazon EC2, fault tolerance, snapshot, backup.

I. INTRODUZIONE

La progettazione di sistemi distribuiti richiede di mantenere uno stato coerente tra repliche distinte in presenza di guasti parziali, assenza di memoria condivisa e comunicazione asincrona. In uno storage chiave-valore replicato, le operazioni confermate devono essere applicate nello stesso ordine da tutte le repliche corrette. Per questo motivo è necessario un protocollo di consenso che coordini scritture, repliche e recovery.

Il progetto B5 richiede la realizzazione di uno storage distribuito capace di replicare lo stato, tollerare guasti e operare in ambiente cloud. In questo lavoro è stato implementato un sistema in Go basato su un protocollo di consenso con log replicato ispirato a Raft. Le scritture sono coordinate dal leader e confermate solo dopo commit su quorum, mirando alla linearizzabilità delle operazioni completate.

Il sistema integra Consensus Node stateful, Client Proxy stateless e Backup Service. Il Proxy fornisce accesso trasparente al cluster, mentre il Backup Service gestisce snapshot e backup persistenti senza partecipare al consenso. La valutazione sperimentale su Amazon EC2 misura latenza, throughput, failover, backup e comportamento con nodo non disponibile.

L'elaborato è organizzato come segue. La Sezione II descrive l'architettura, la Sezione III i pattern architetturali, la Sezione IV l'implementazione, la Sezione V il deployment, la Sezione VI i risultati sperimentali, mentre le Sezioni VII e VIII discutono limitazioni e conclusioni.

II. ARCHITETTURA DEL SISTEMA

L'architettura del sistema, mostrata in Fig. 1, è organizzata attorno a tre componenti principali: *Consensus Node*, *Client Proxy Service* e *Snapshot & Backup Service*. La separazione tra questi componenti distingue le responsabilità stateful, legate alla replica e alla persistenza dello stato, da quelle stateless, legate al routing delle richieste e alle operazioni amministrative.

Il sistema rimuove la dipendenza da un database centralizzato replicando lo stato applicativo tramite un protocollo di consenso ispirato a Raft. Il client invia le richieste al Client Proxy, che individua il leader corrente e inoltra le operazioni al nodo corretto. Il leader replica quindi le entry di log sui follower e conferma le scritture solo dopo il raggiungimento di un quorum, pari a $\lfloor N/2 \rfloor + 1$ in un cluster di N nodi. Questo modello consente di mantenere consistenti le operazioni confermate, poiché ogni scrittura applicata alla state machine è stata accettata da una maggioranza del cluster.

A. Consensus Node

I Consensus Node costituiscono il nucleo stateful del sistema. Ogni nodo partecipa al protocollo di consenso e può assumere dinamicamente il ruolo di follower, candidate o leader. Il leader riceve le scritture, crea le entry di log e coordina la replica verso i follower. Una entry viene applicata alla state machine chiave-valore solo dopo il commit su quorum.

Ogni nodo mantiene persistentemente termine corrente, voto espresso, log delle operazioni, state machine, snapshot locali e metadati di recovery. La durabilità è garantita tramite Write-Ahead Log, mentre gli snapshot locali consolidano lo stato e riducono il costo del recupero. Durante i test su EC2 sono stati osservati file persistenti separati per ciascun nodo, come `node-X_wal.log`, `node-X_snapshot.json` e `node-X_state.json`.

B. Client Proxy Service

Il Client Proxy Service è un componente stateless che rappresenta il punto di ingresso unico per i client esterni. Come indicato in Fig. 1, il client non comunica direttamente con i Consensus Node, ma invia sempre le richieste al Proxy, che si occupa di instradarle verso il leader corrente.

Il Proxy usa la RPC `GetLeader` e i `leader_hint` restituiti dai follower per aggiornare il routing. Inoltre integra retry, backoff e Circuit Breaker, così da evitare che nodi non disponibili degradino l'intero sistema. Nel deployment Docker

Architettura logica del sistema B5

Client Proxy stateless, Consensus Node stateful e Backup Service stateless

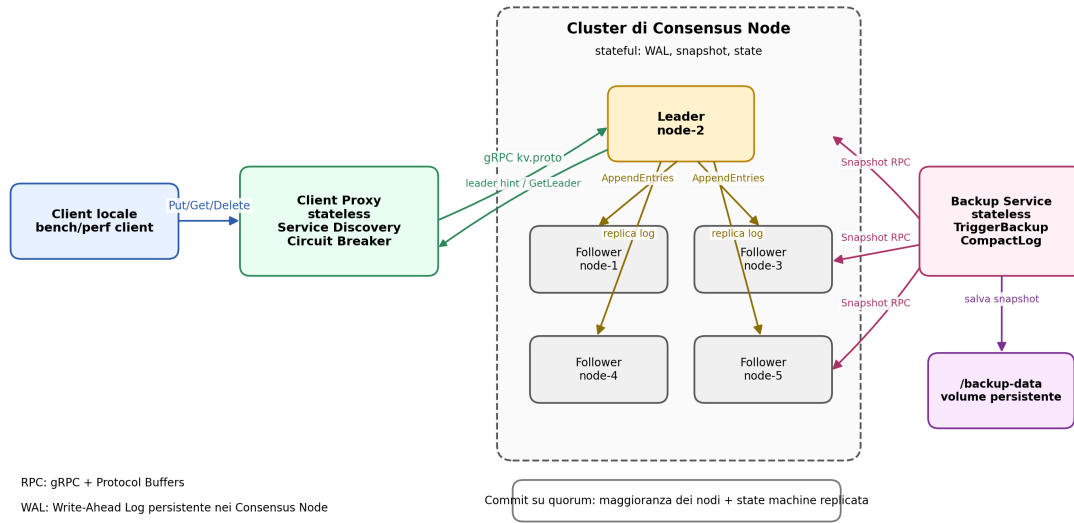


Figura 1. Architettura logica del sistema: il Client Proxy espone un punto di ingresso unico, i Consensus Node replicano lo stato tramite consenso e il Backup Service coordina snapshot e backup persistenti.

Compose espone anche l'endpoint HTTP `/health`, usato per gli healthcheck del container.

C. Snapshot & Backup Service

Lo Snapshot & Backup Service è il componente amministrativo del sistema. È stateless rispetto al consenso e allo stato applicativo: non partecipa a Raft e non mantiene una replica attiva della state machine. Tuttavia agisce come gestore esterno di artefatti persistenti, salvando gli snapshot ricevuti nel volume `/backup-data`. Non si trova nel percorso critico delle normali operazioni client, ma comunica con i Consensus Node per forzare snapshot, scaricare lo stato e richiedere la compattazione logica dei log remoti.

Il servizio espone la RPC `TriggerBackup`. Nei test sperimentali, fermando un nodo in un cluster da 5 nodi, il backup è stato comunque accettato e completato scaricando 4 snapshot su 5, confermando la tolleranza a nodi non disponibili.

D. Modello di comunicazione

Tutte le interazioni tra componenti sono basate su gRPC e Protocol Buffers. I contratti sono divisi in tre domini: `consensus.proto` per le RPC interne al consenso, `kv.proto` per le operazioni chiave-valore e `backup.proto` per le operazioni amministrative del Backup Service.

Questa modularità mantiene chiari i confini tra i sottosistemi: i Consensus Node gestiscono stato e consenso, il Client Proxy gestisce discovery e routing, mentre il Backup Service gestisce snapshot e backup. L'architettura rispetta quindi il principio di separazione delle responsabilità ed evita componenti monolitici.

III. PATTERN ARCHITETTURALI E COMUNICAZIONE

Il sistema utilizza tre meccanismi principali: gRPC con Protocol Buffers per definire interfacce tipizzate, Client-side

Service Discovery nel Proxy per individuare dinamicamente il leader e Circuit Breaker per isolare nodi non disponibili. Questi pattern risolvono problemi complementari: coordinare servizi separati, evitare che il client dipenda dalla posizione del leader e prevenire cascading failure.

Come mostrato in Fig. 2, il client invia la richiesta al Proxy. Se il leader è noto, il Proxy inoltra la richiesta al nodo corretto; altrimenti esegue discovery tramite `GetLeader`. Se una richiesta raggiunge un follower, il `leader_hint` consente di aggiornare la cache. La chiamata verso ciascun nodo è protetta da un Circuit Breaker indipendente.

A. gRPC e Protocol Buffers

La comunicazione usa gRPC e Protocol Buffers per avere contratti espliciti e fortemente tipizzati. I file `.proto` sono separati per dominio: `consensus.proto` per consenso e replica, `kv.proto` per le operazioni client-facing e `backup.proto` per snapshot e backup. Il vantaggio è la coerenza tra client e server tramite stub Go generati automaticamente; il limite è la necessità di rigenerare gli stub quando cambiano le interfacce.

B. Client-side Service Discovery

Il leader può cambiare dopo crash o timeout, quindi non può essere configurato staticamente nel client. Il Proxy implementa una Client-side Service Discovery: mantiene una cache del leader, interroga i nodi tramite `GetLeader` e usa il `leader_hint` per aggiornare il routing. Il vantaggio è la trasparenza per il client, che vede un unico endpoint stabile; il limite è che il Proxy diventa un componente critico, anche se stateless e replicabile.

Service Discovery e Circuit Breaker nel flusso Put

Il Proxy individua il leader e protegge le chiamate ai nodi con circuiti indipendenti

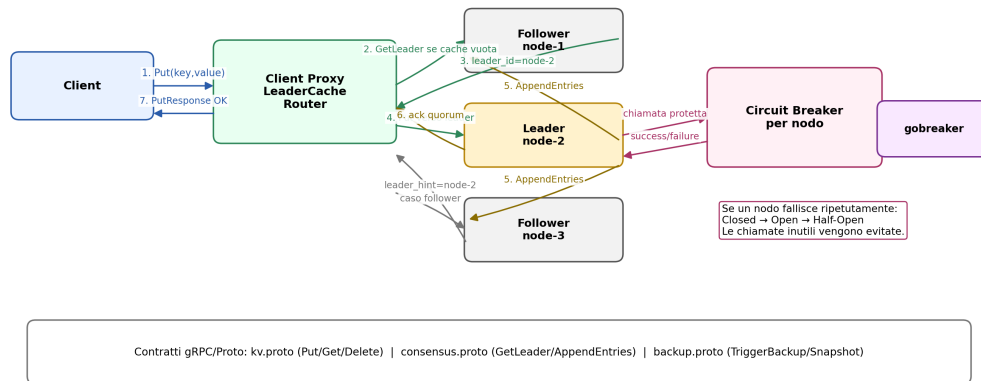


Figura 2. Flusso di una richiesta `Put`: il Proxy scopre il leader, aggiorna la cache e protegge le chiamate ai nodi tramite Circuit Breaker.

C. Circuit Breaker

Il Circuit Breaker evita chiamate ripetute verso nodi non disponibili, preservando le risorse del Proxy e del Backup Service. Ogni nodo è associato a un circuito indipendente implementato con `gobreaker`. Se un nodo fallisce ripetutamente, il circuito passa temporaneamente in stato *Open*; successivamente può passare in *Half-Open* per verificare il recupero. Nei test, con un nodo offline, il Backup Service ha completato il backup con `downloaded_snapshots=4`, confermando l'isolamento del guasto.

IV. IMPLEMENTAZIONE

Il sistema è implementato in Go, scelto per il supporto nativo alla concorrenza tramite goroutine, la compilazione in binari statici e l'integrazione con gRPC. Le dipendenze sono gestite tramite `go.mod` e includono `google.golang.org/grpc`, `google.golang.org/protobuf` e `github.com/sony/gobreaker/v2`. Gli stub Go sono generati dai file `.proto`, mantenendo allineati client e server rispetto ai contratti RPC.

A. Consensus Node

Il Consensus Node è stato organizzato separando consenso, storage e persistenza. La logica di consenso gestisce termini, voti, heartbeat, timer di elezione randomizzati e replica tramite RPC ispirate ad `AppendEntries`. Lo storage mantiene una state machine deterministica basata su mappa chiave-valore. Le operazioni modificanti, come `Put` e `Delete`, diventano entry del log e vengono applicate allo stato solo dopo il commit su quorum.

B. Persistenza, WAL e snapshot

La persistenza usa un Write-Ahead Log event-based: ogni modifica viene scritta in un file append-only prima di essere considerata durevole. Nei container su EC2 sono stati osservati file come `node-1_wal.log`, `node-1_snapshot.json`

e `node-1_state.json`, confermando che lo stato non è solo volatile.

Gli snapshot JSON rappresentano checkpoint della state machine e possono essere forzati dal Backup Service. La compattazione implementata è logica: lo stato viene consolidato in snapshot e le entry possono essere ridotte a livello di stato in memoria, ma il file WAL fisico non viene riscritto né troncato immediatamente. Nei test il WAL resta intorno a 7.2 MB per nodo, aspetto discusso nelle limitazioni.

C. Container e benchmark

Ogni componente principale è distribuito come container separato. Docker Compose configura rete, volumi, porte e variabili d'ambiente per cluster da 3, 5 e 7 nodi. Per i test sono stati implementati client dedicati: `perf-client` per latenza e throughput, `failover-client` per crash del leader e `backup-benchmark-client` per snapshot e backup. I risultati sono esportati in CSV e trasformati in grafici tramite script Python.

V. DEPLOYMENT SU AMAZON EC2

A. Ambiente cloud

Il deploy finale è stato eseguito su Amazon EC2 nell'ambiente AWS Academy Learner Lab. Il sistema è stato installato su una singola istanza `t3.small` con Amazon Linux 2023, configurata con Docker, Docker Compose, Git e la toolchain necessaria alla compilazione dei servizi.

Il Security Group espone le sole porte necessarie ai test: `8080/tcp` per il Client Proxy gRPC, `8081/tcp` per l'endpoint `/health`, `9090/tcp` per il Backup Service e `22/tcp` per SSH. I client esterni interagiscono con Proxy e Backup Service, mentre i Consensus Node comunicano principalmente nella rete Docker interna.

B. Orchestrazione

Docker Compose è stato usato per definire servizi, rete, volumi e variabili d'ambiente. Sono presenti configurazioni distinte per cluster da 3, 5 e 7 nodi, poiché ogni Consensus Node richiede identificativo, peer list e volume dedicato. La rete bridge interna consente ai container di comunicare tramite nomi di servizio, come `node-1`, `client-proxy` e `backup-service`, senza dipendere da IP statici.

C. Benchmark remoti

I benchmark ufficiali sono stati eseguiti dal PC locale verso l'indirizzo IPv4 pubblico dell'istanza EC2. Questa configurazione misura il percorso end-to-end:

PC → Internet → Security Group → EC2 →
Proxy → Cluster

Le misure includono quindi latenza WAN, firewall AWS e routing interno Docker. Prima dei test è stato verificato l'endpoint `/health`; latenza, throughput, failover e backup sono stati raccolti tramite client dedicati eseguiti localmente.

VI. VALUTAZIONE SPERIMENTALE

La valutazione sperimentale verifica il comportamento del sistema nell'ambiente descritto in Sezione V. Prima delle misure è stata eseguita una fase di warm-up di 30 Put e 30 Get, esclusa dai risultati, per ridurre l'impatto di apertura delle connessioni gRPC, prima discovery del leader, cache vuote e scheduling iniziale dei container.

A. Scalabilità e performance

Il primo esperimento misura l'impatto del numero di nodi sulle prestazioni. Sono state testate configurazioni con 3, 5 e 7 Consensus Node; per ciascuna configurazione sono state eseguite 300 Put e 300 Get. La Tab. I riassume latenza media, P99 e throughput.

Tabella I
PRESTAZIONI END-TO-END DAL PC LOCALE VERSO EC2.

Nodi	Op.	Avg ms	P99 ms	ops/s
3	Get	107.05	109.46	9.37
5	Get	121.61	274.56	8.25
7	Get	106.29	112.14	9.44
3	Put	109.81	124.46	9.14
5	Put	123.16	152.66	8.15
7	Put	110.88	119.13	9.05

Le latenze medie restano tra 100 e 125 ms. Le Put sono leggermente più costose delle Get perché richiedono scrittura su WAL e replica su quorum. Il throughput resta vicino a 9 ops/s nelle configurazioni principali.

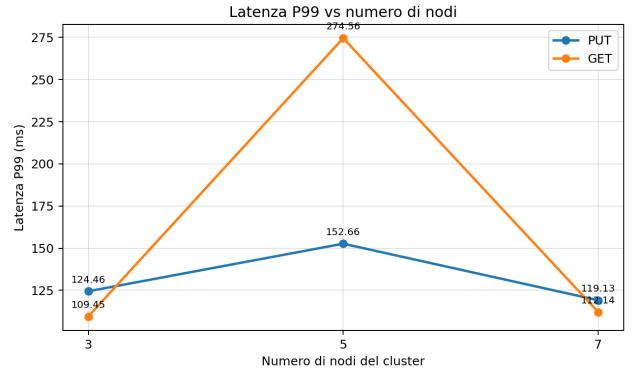


Figura 3. Latenza P99 al variare del numero di nodi.

La Fig. 3 evidenzia outlier nella configurazione a 5 nodi, soprattutto sulle Get. Poiché il success rate è pari al 100% in tutte le configurazioni, tali picchi sono interpretati come jitter infrastrutturale e non come errore del protocollo.

B. Tolleranza ai guasti

Il secondo esperimento valuta il recupero dopo crash del leader su un cluster da 5 nodi. In 10 trial il sistema ha sempre rieletto un leader ed è tornato a servire richieste. Il tempo medio di rilevazione del nuovo leader è stato 1270.3 ms, con mediana 1110 ms; il downtime medio percepito dal client è stato 1893.6 ms, con mediana 1234.5 ms.

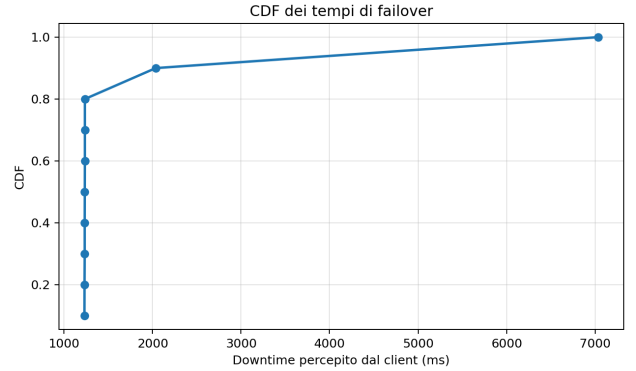


Figura 4. CDF del downtime percepito dal client durante crash del leader.

La Fig. 4 mostra che l'80% dei failover recupera entro 1.24 s e il 90% entro 2.04 s. L'outlier massimo non ha compromesso la correttezza: dopo ogni rielezione il sistema ha ripreso a servire scritture.

C. Backup e fault isolation

Il Backup Service è stato testato con dataset da 100, 300 e 600 chiavi. In tutti i casi il backup è stato accettato e sono stati scaricati 5 snapshot su 5 nodi. Gli snapshot in `/backup-data` crescono coerentemente con il dataset: circa 7.6 KB, 27.5 KB e 67.3 KB.

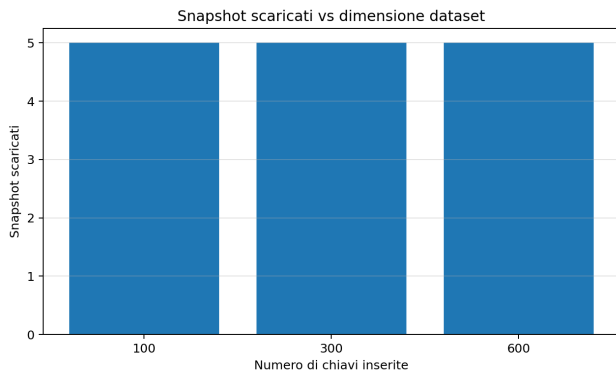


Figura 5. Snapshot scaricati dal Backup Service al variare del dataset.

Per validare il Circuit Breaker è stato fermato un nodo in un cluster da 5 nodi e invocato un nuovo backup. Il servizio ha risposto con `accepted=true` e `downloaded_snapshots=4`, dimostrando che l'indisponibilità di un nodo viene isolata senza bloccare l'intera operazione.

VII. LIMITAZIONI

A. Persistenza e compattazione

La compattazione implementata è logica: il sistema produce snapshot e consolida la state machine, ma il file WAL fisico rimane append-only. Nei test il file `node-X_wal.log` resta intorno a 7.2 MB per nodo; in produzione servirebbe una truncation fisica dopo il checkpoint.

B. Ottimizzazioni del protocollo

Il sistema privilegia la correttezza rispetto alla massima ottimizzazione. Le scritture sono confermate su quorum e le letture sono gestite in modo conservativo; non sono implementate ottimizzazioni come *ReadIndex* o lease-based reads. Anche i timeout di elezione sono conservativi, riducendo instabilità e split-vote ma aumentando il tempo di failover rispetto a configurazioni più aggressive.

C. Misure e guasti

La precisione dei tempi di failover è limitata dal polling del `failover-client`, che interroga il Proxy a intervalli di circa 100 ms. Inoltre, i test hanno coperto guasti fail-stop ottenuti arrestando container Docker; non sono stati simulati partizionamenti di rete, nodi intermittenti o stati dell'ordine di molti GB.

VIII. CONCLUSIONI

A. Obiettivi raggiunti

Il progetto ha realizzato uno storage chiave-valore distribuito, replicato e deployabile in cloud, soddisfacendo i requisiti principali del bando B5. La dipendenza da un database centralizzato è stata rimossa tramite un protocollo di consenso con log replicato ispirato a Raft, basato su leader election, commit su quorum e persistenza locale.

B. Risultati principali

La valutazione sperimentale su Amazon EC2 ha confermato il funzionamento end-to-end: success rate del 100% nei benchmark con 3, 5 e 7 nodi, throughput vicino a 9 ops/s, rielezione del leader in circa 1.27 s in media e downtime mediano pari a circa 1.23 s. Il Backup Service ha inoltre completato backup parziali con 4 snapshot su 5 in presenza di un nodo offline.

C. Sviluppi futuri

Il sistema costituisce una base solida per uno storage distribuito resiliente e containerizzato. Le principali evoluzioni future riguardano truncation fisica del WAL, TLS e autenticazione, cluster multi-istanza e validazione con dataset e carichi concorrenti più elevati.

RIFERIMENTI BIBLIOGRAFICI

- [1] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," in *Proc. USENIX Annual Technical Conference*, Philadelphia, PA, USA, 2014, pp. 305–320.
- [2] M. P. Herlihy and J. M. Wing, "Linearizability: A correctness condition for concurrent objects," *ACM Transactions on Programming Languages and Systems*, vol. 12, no. 3, pp. 463–492, 1990.
- [3] M. Fowler, "CircuitBreaker," Martin Fowler's Blog, Mar. 6, 2014. [Online]. Available: <https://martinfowler.com/bliki/CircuitBreaker.html>
- [4] C. Richardson, "Circuit Breaker Pattern," Microservices.io. [Online]. Available: <https://microservices.io/patterns/reliability/circuit-breaker.html>
- [5] gRPC Authors, "gRPC Documentation – Go." [Online]. Available: <https://grpc.io/docs/languages/go/>
- [6] Sony, "gobreaker: Circuit breaker implementation in Go." [Online]. Available: <https://github.com/sony/gobreaker>
- [7] Docker Inc., "Docker Compose Documentation." [Online]. Available: <https://docs.docker.com/compose/>
- [8] Amazon Web Services, "Amazon EC2 Documentation." [Online]. Available: <https://docs.aws.amazon.com/ec2/>